IndicDoc AI

Finding the structure and reading the text of Indian-language documents

Project report · 06 October 2026

A plain-language account of what this project does, how it works, how it was trained and tested, and how well it performs. Every number in this report was produced by the project's own code on real IndicDLP pages.

1. The project in one paragraph

IndicDoc AI takes a picture or PDF of a document page (a textbook, a newspaper, a government notice, a question paper) in any of 12 Indian languages. It does two things. First it finds the **layout**: it draws a box around every part of the page (each paragraph, heading, table, figure, page number, list or question) and gives each box one of 42 labels. Then it **reads the text** inside those boxes (OCR) and returns the page as an organised document. The project compares three layout models on the same 682 test pages that none of them was trained on: two trained here on a laptop (Faster R-CNN and YOLOv8s), and DocLayout-YOLO, published by the dataset's authors and trained on the full dataset. The authors' model is the best by far, at **0.759 mAP@50** (explained in section 6), so it is the one the app uses by default. Everything ships as a web app and a REST API in one Docker image that also works without internet access.

2. The problem

Before a computer can usefully read a document, it has to understand its *layout*: which text is a heading, which is body text, where the table is, which number is the page number. People do this at a glance; a computer sees only pixels. This step is called **document layout analysis**, and it decides how well everything after it works. For example, if OCR reads a two-column page straight across, the result is nonsense.

Most layout tools are built and tested on English documents. Indian documents use many scripts (Devanagari, Bengali, Tamil, Telugu, Kannada, Malayalam, Gujarati, Gurmukhi, Odia, Latin) and many formats: dense multi-column newspapers, question papers with nested sub-questions, legal acts with numbered clauses.

> **Input:** one page image, or one page of a PDF.
> **Output:** boxes with labels (e.g. `paragraph`, `table`, `section-title`) and confidence scores, plus the text of every box, in reading order, as Markdown and JSON.

3. The data

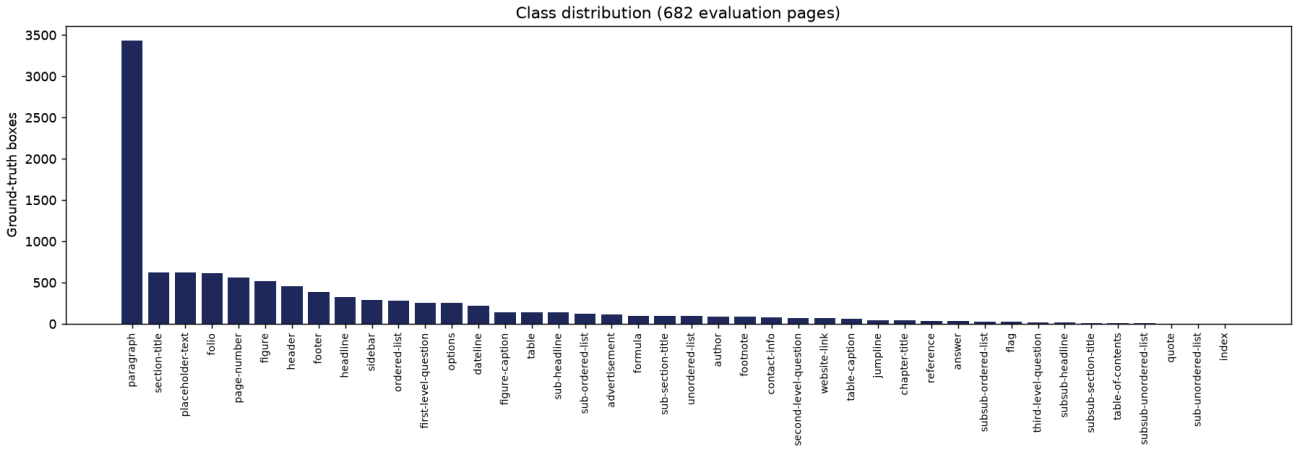
The project uses **IndicDLP** (Nath et al., ICDAR 2025; AI4Bharat, IIT Madras and IIIT Hyderabad): 119,806 human-annotated pages in 12 languages and 12 document types, labelled with 42 layout classes. It is listed on India's AIKosh platform and downloaded from Hugging Face.

The full dataset is about 91 GB, too much to train on with a laptop in a few hours. So for its own two models the project builds a **balanced subset** with roughly the same number of pages from every language. It uses the dataset's own train / validation / test split, so test pages are never seen during training.

Split	Pages	Labelled boxes	Used for
Train	2293	33667	Learning our two models' weights
Validation	342	5815	Choosing checkpoints and confidence thresholds
Test	682	10513	Final, unbiased scores for all three models

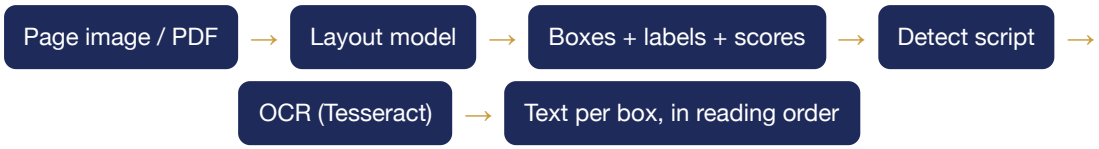
Language	train pages	val pages	test pages
Assamese	129	30	60
Bengali	200	30	60
English	188	30	60
Gujarati	200	30	60
Hindi	200	21	22
Kannada	200	21	60
Malayalam	200	30	60
Marathi	200	30	60
Odia	200	30	60
Punjabi	188	30	60
Tamil	200	30	60
Telugu	188	30	60

The labels are very unevenly spread: `paragraph` alone is 31% of all training boxes, while some labels appear only a handful of times. Rare labels are much harder to learn.



How often each label appears in the test pages.

4. How it works



The three layout models

Faster R-CNN (our baseline) works in two steps. A *region proposal network* suggests places on the page that might contain something, then a second network looks closely at each one, decides its label and tightens the box. ResNet-50 backbone with a feature pyramid; 41.5 million parameters.

YOLOv8s (our proposed model) ("You Only Look Once") does everything in a single pass over the whole page. It has 11.2 million parameters, about a quarter of Faster R-CNN, so it is smaller and faster.

DocLayout-YOLO (IndicDLP authors) is a YOLO-family detector built specially for documents (Zhao et al., 2024): it was pre-trained on 300,000 synthetic document pages, then fine-tuned by the IndicDLP authors on the *full* IndicDLP training set (~96K pages, 100 epochs, 1024-pixel images, on server GPUs). 20.0 million parameters. It was **not trained in this project**: it was downloaded from the authors' public release (Zenodo, CC-BY 4.0) and evaluated here on exactly the same test pages as our models.

Transfer learning

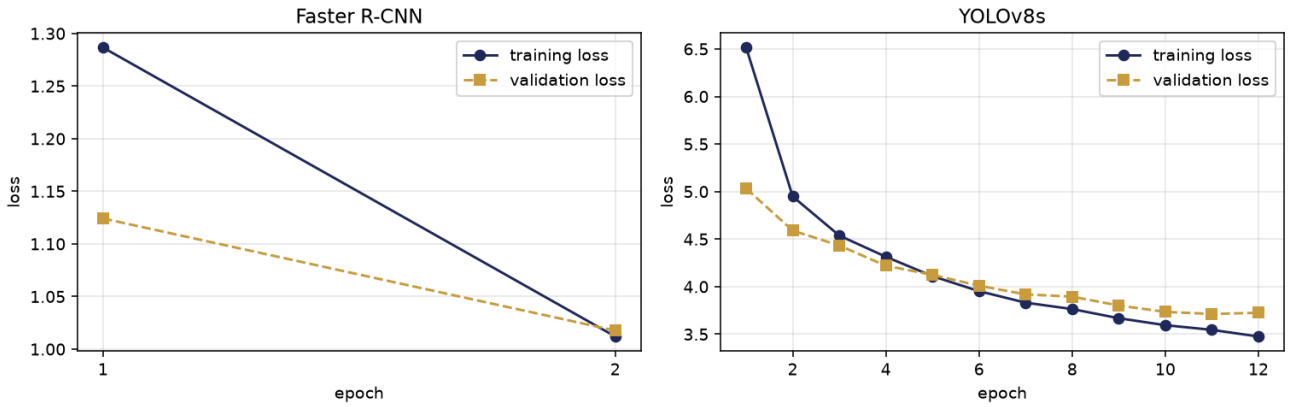
None of the models starts from zero. They begin with weights already trained on other images (COCO everyday photos for ours, synthetic documents for DocLayout-YOLO), so they already know how to see edges, shapes and text-like textures. Their final layer is replaced with one for the 42 document labels and the network is then *fine-tuned*.

5. Training our two models

	Faster R-CNN	YOLOv8s
Hardware	Apple M1 Pro GPU (MPS)	Apple M1 Pro GPU (MPS)
Epochs	2	12
Batch size	4	8
Training time	76 min	60 min
Optimizer	SGD, lr 0.005, momentum 0.9, cosine schedule	Chosen automatically by Ultralytics
Augmentation	colour jitter, blur (no flips)	mosaic, HSV, translate/scale (no flips)
Best checkpoint	lowest validation loss	best validation mAP

During training the model predicts boxes on training pages, measures how wrong they are (the *loss*) and adjusts its weights to be a little less wrong. One *epoch* is one pass over all training pages. After every

epoch the model is scored on the validation pages and the best version is kept. Falling curves mean the models were learning; they had not stopped improving, so longer training would help.



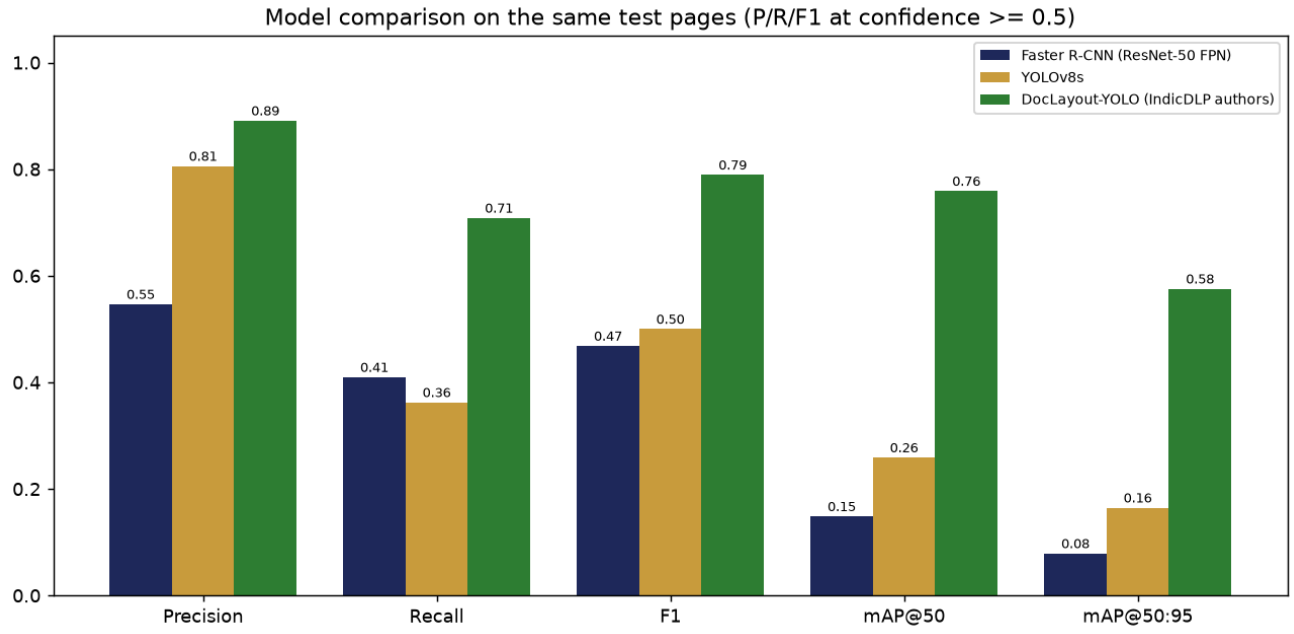
Loss per epoch (lower is better). Training loss in blue, validation loss in gold.

6. Layout results

How to read the scores. A predicted box counts as correct if it has the right label and overlaps the true box enough: **IoU** (overlap ÷ combined area) of at least 0.5. **Precision** = of the boxes the model drew, the share that were right. **Recall** = of the real elements on the page, the share the model found. **mAP@50** = the standard overall score for detectors (0 to 1): precision averaged over every recall level and over all labels, at IoU 0.5. **mAP@50:95** is stricter: it also demands tightly fitting boxes. The **confidence threshold** decides which boxes are shown. Each model has a *recommended* threshold, the one with the best F1 on the validation pages (never tuned on the test pages). Because finding every part matters more than a tidy page for this project, the app's default is a second, **"find every part"** threshold: the lowest one that still kept at least 70% of boxes correct on the validation pages, and never higher than the balanced one. In both modes, a box that the model proposes under several labels is shown once, with its most likely label.

Metric (same 682 test pages)	DocLayout-YOLO (authors)	YOLOv8s (ours)	Faster R-CNN (ours)
mAP@50	0.759	0.259	0.148
mAP@50:95	0.575	0.165	0.079
mAP@75	0.633	0.172	0.075
Recommended confidence (best F1 on validation)	0.35	0.25	0.45
Precision at recommended confidence	0.848	0.603	0.510
Recall at recommended confidence	0.775	0.508	0.435
F1 at recommended confidence	0.810	0.551	0.469
"Find every part" confidence (validation: precision >= 0.70, at most the balanced value)	0.15	0.25	0.45
Precision, find every part	0.742	0.603	0.510
Recall, find every part	0.851	0.508	0.435
Average IoU of correct boxes	0.899	0.885	0.821
Time per page (s, Apple M1 Pro GPU)	0.150	0.024	0.202
Model size (MB)	161.5	22.5	166.6

The authors' DocLayout-YOLO is far ahead: mAP@50 0.759 against 0.259 for our YOLOv8s and 0.148 for our Faster R-CNN, and F1 0.810 at its recommended threshold. That is what a document-specialised model trained on ~40 times more pages for 100 epochs buys. Between our two models, YOLOv8s wins on accuracy and is 8.6x faster than Faster R-CNN (0.02 vs 0.20 s per page), with 22 MB of weights instead of 167 MB. DocLayout-YOLO works on larger 1024-pixel images and needs 0.15 s per page on the same machine.

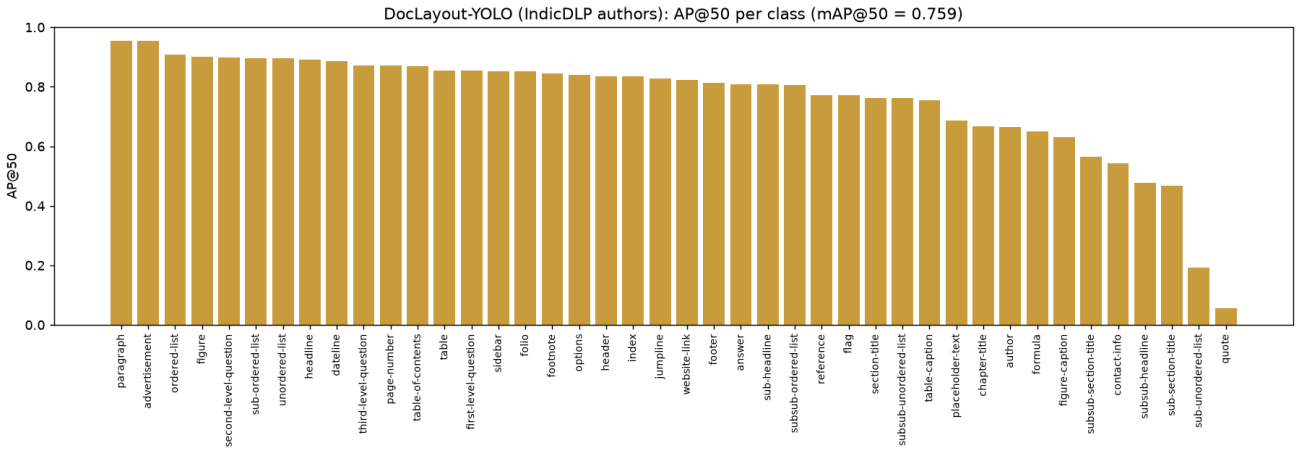


All three models on the same 682 test pages.

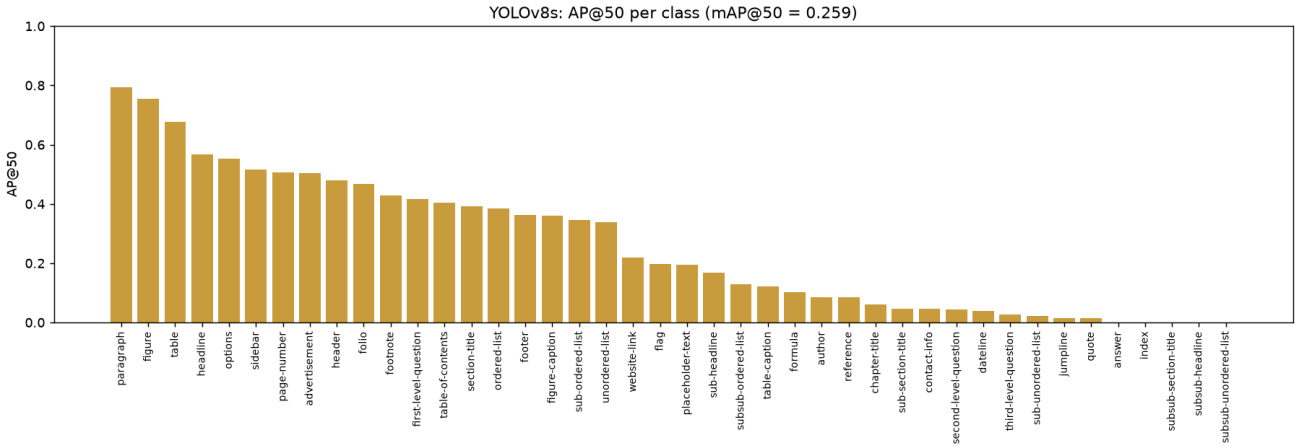
Is the score correct? The evaluator uses `pycocotools`, the reference COCO implementation, and is tested on hand-worked examples. Two independent checks agree with it. Ultralytics' own AP code, run on the same YOLOv8s detections, gives 0.258 against 0.259 here. And the authors' model scores 0.759 on our test pages, in line with the about 0.735 the IndicDLP paper reports on its full test set.

Which layout elements are easy, which are hard

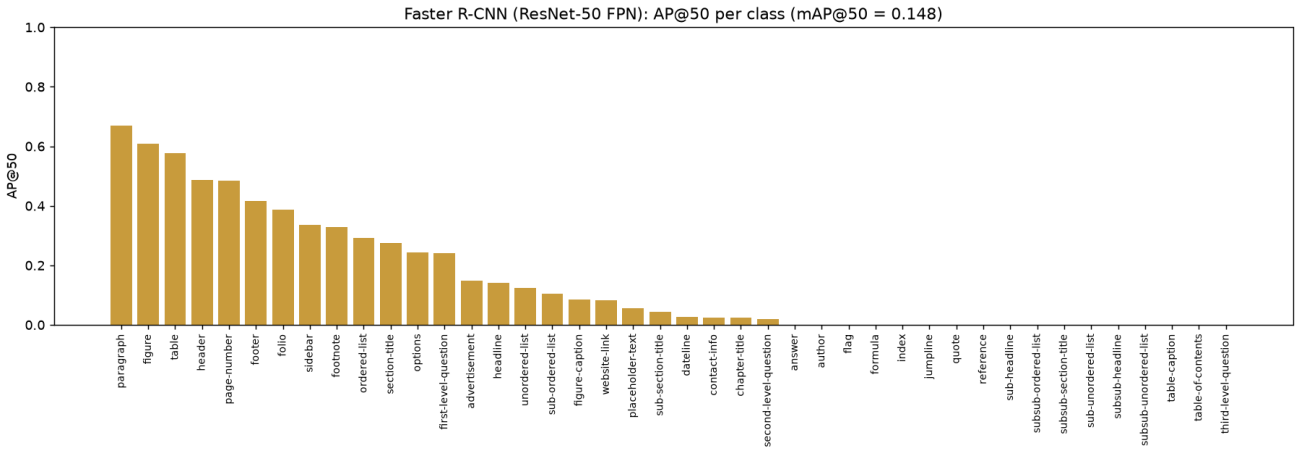
For DocLayout-YOLO, the easiest labels are paragraph (0.95), advertisement (0.95), ordered-list (0.91), figure (0.90). The hardest are subsub-headline (0.48), sub-section-title (0.47), sub-unordered-list (0.19), quote (0.06): rare, or easily confused with similar labels (for example the three levels of nested lists). All 42 labels occur in the test pages.



AP@50 per label, DocLayout-YOLO (authors).



AP@50 per label, YOLOv8s (ours).



AP@50 per label, Faster R-CNN (ours).

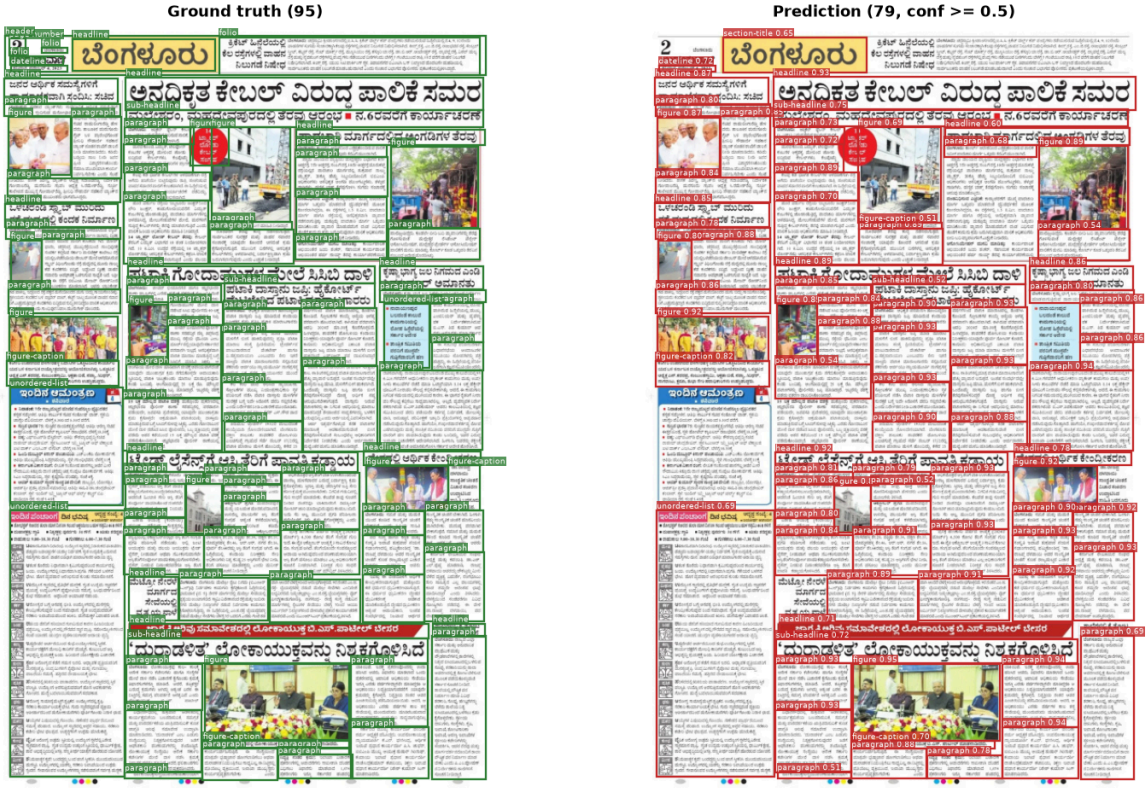
Results per language

Language	Test pages	DocLayout-YOLO (authors)	YOLOv8s (ours)	Faster R-CNN (ours)
Assamese	60	0.835	0.390	0.273
Bengali	60	0.757	0.303	0.199
English	60	0.768	0.315	0.186
Gujarati	60	0.779	0.308	0.190
Hindi	22	0.879	0.368	0.230
Kannada	60	0.757	0.355	0.186
Malayalam	60	0.776	0.432	0.270
Marathi	60	0.829	0.383	0.246
Odia	60	0.793	0.339	0.234
Punjabi	60	0.822	0.329	0.165
Tamil	60	0.807	0.339	0.223
Telugu	60	0.814	0.357	0.195

For DocLayout-YOLO, scores range from 0.879 (Hindi) to 0.757 (Bengali): it works well in every language. Most languages have 60 test pages (Hindi has 22), so differences of a few points are within normal variation. Languages can score above the overall figure: mAP averages over the labels present, and each language's pages contain only part of the 42 labels.

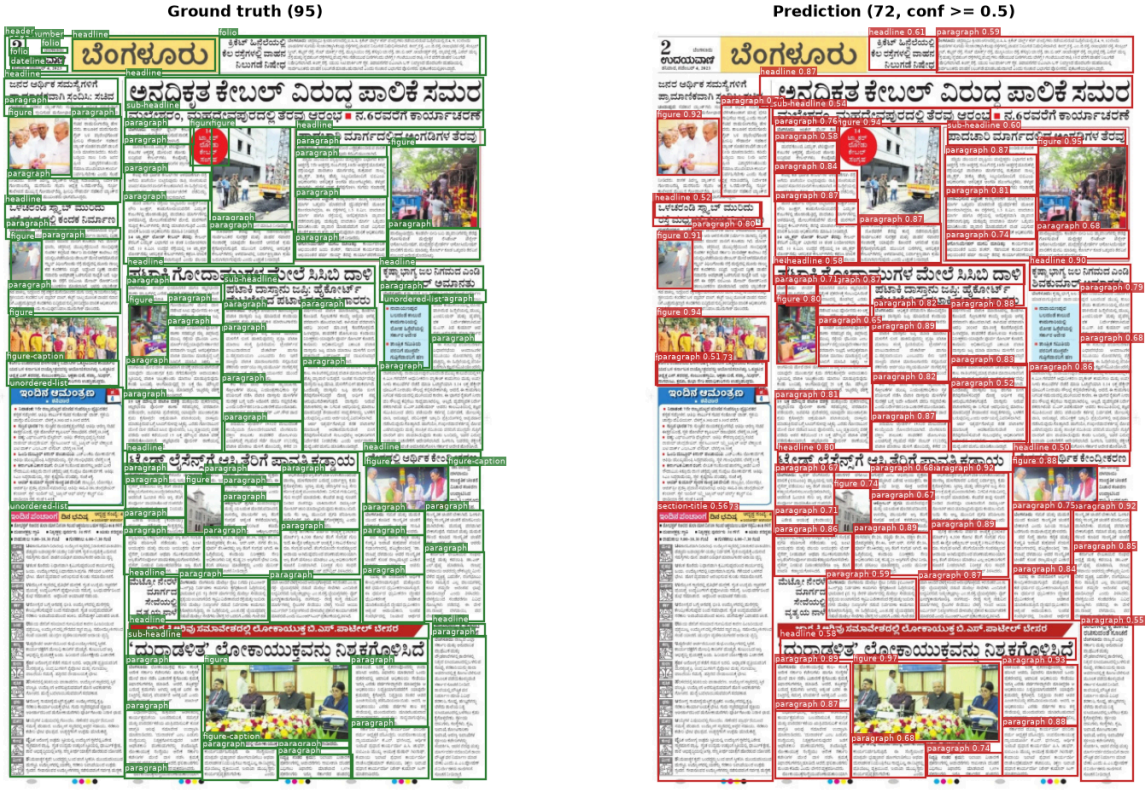
What the predictions look like

DocLayout-YOLO (IndicDLP authors) on np_kn_000364_1.png



DocLayout-YOLO on a test page: true layout (left, green) vs prediction (right, red).

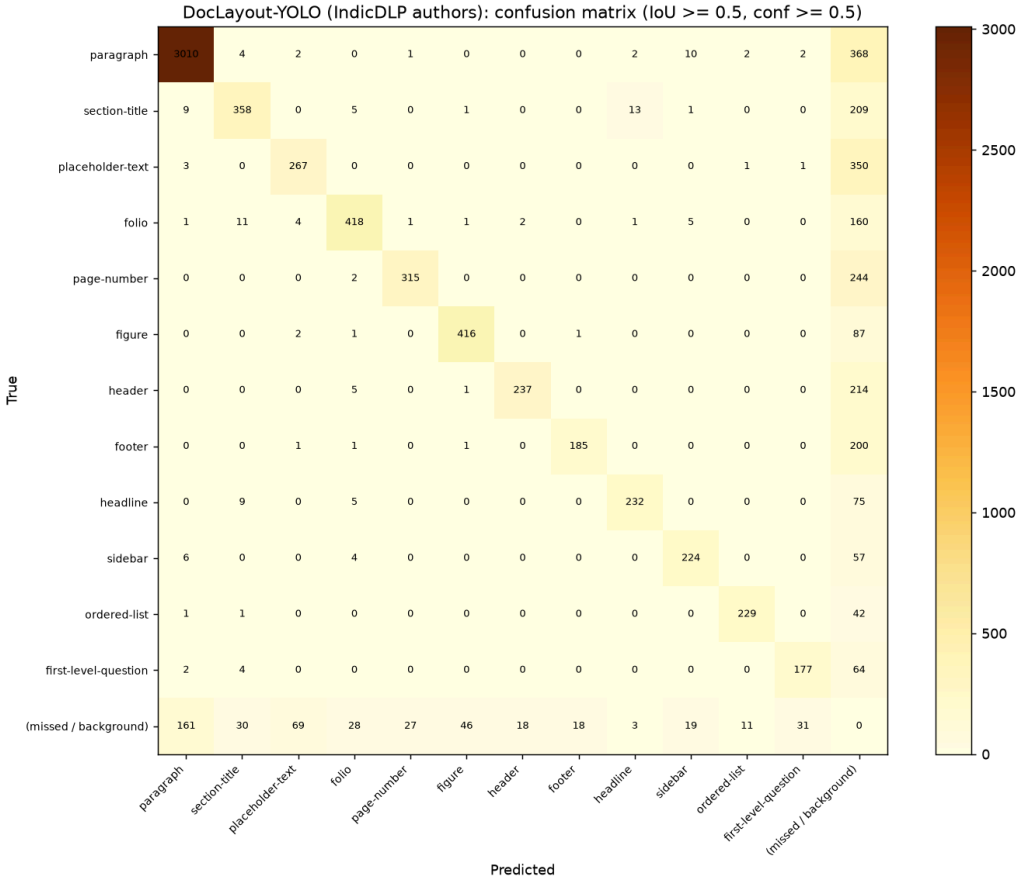
YOLOv8s on np_kn_000364_1.png



YOLOv8s on a test page: true layout (left, green) vs prediction (right, red).

Where the best model goes wrong

The confusion matrix shows, for the most common labels, which true label (rows) was predicted as which label (columns). The last row and column count missed elements and false alarms.



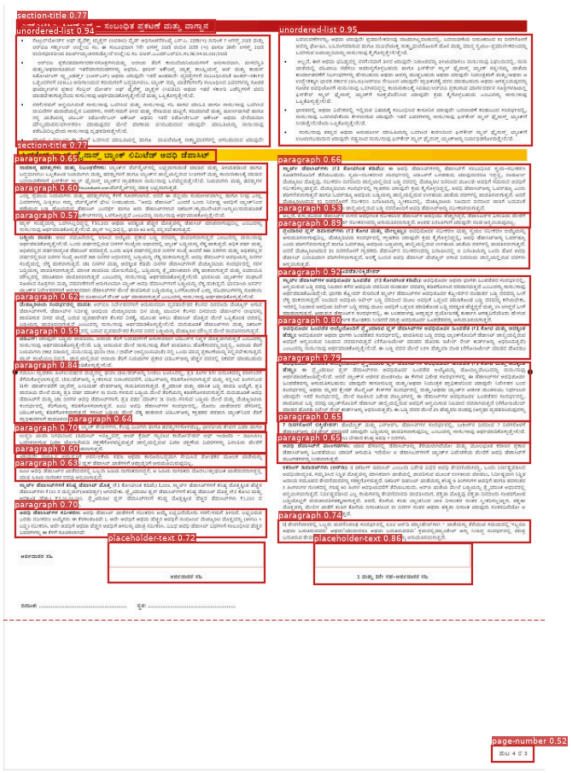
Confusion matrix of DocLayout-YOLO on the test pages.

Most false detections: 23 on fm_kn_000234_1.png

Ground truth (15)



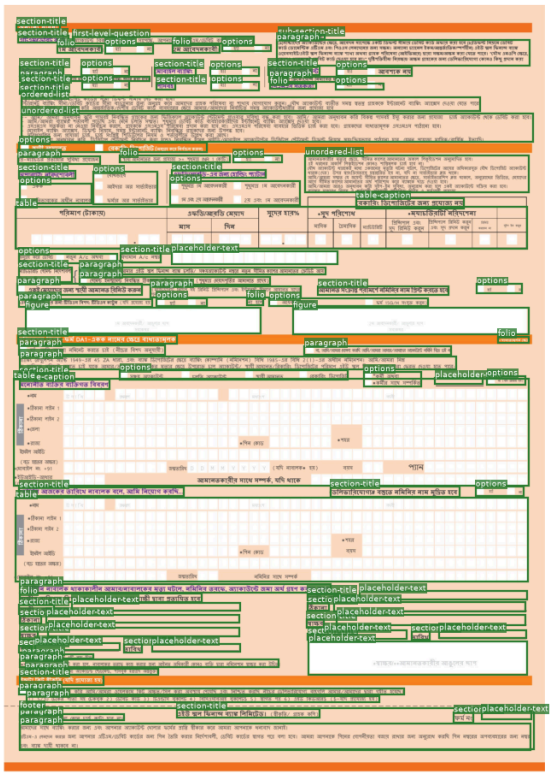
Prediction (27, conf >= 0.5)



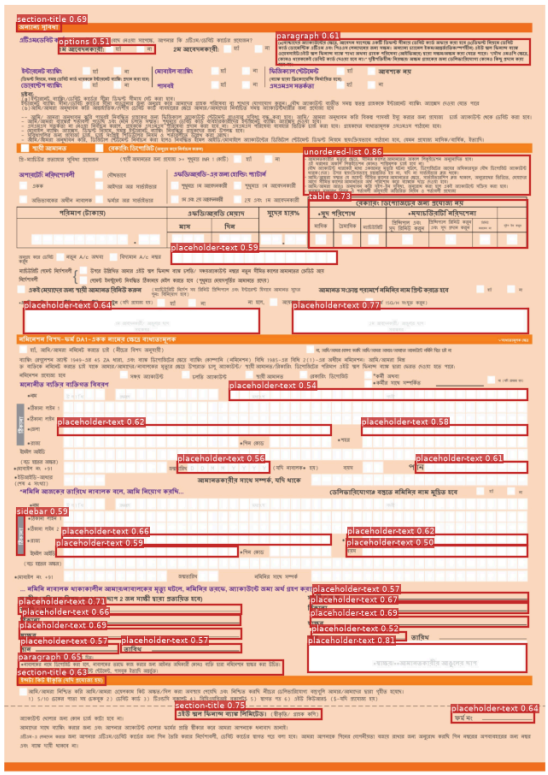
DocLayout-YOLO: the test page with the most false detections.

Most missed elements: 90 on fm_bn_000041_0.png

Ground truth (105)



Prediction (32, conf >= 0.5)



DocLayout-YOLO: the test page with the most missed elements.

7. Reading the text (OCR)

Finding boxes is half the job; the other half is reading what is inside them. The project uses **Tesseract** (Google's open-source OCR engine) with its best models for all 12 languages.

- **One read per page.** Tesseract reads the whole page once and reports every word with its position. Each word is then placed in the detected box that contains it. This is much faster than reading 60 boxes separately, and Tesseract reads whole lines more accurately than tight crops. 94% of the words it finds fall inside a detected box, so the layout model covers almost all the text.
- **Choosing the language automatically.** Tesseract's built-in script detector proved unreliable on these pages (it got only 70 of 120 test pages right, and none of the Gujarati, Odia or Punjabi ones). The project instead reads one text block with each script's model and keeps the most confident reading, counting only characters that belong to that script. On 120 test pages (10 per language) this picks the right script for **106 (88%)**. In the app the language can also be chosen by hand.
- **Nothing is lost.** Words that fall in no detected box are not dropped: they are grouped into extra "unlabelled text" blocks, so the extracted document contains all the text even where the layout model missed an element.
- **Organised output.** Boxes are put in reading order (top to bottom, left to right) and written as Markdown: headings become headings, lists become lists, tables stay as tables, figures and formulas are marked `[figure]` / `[formula]` (OCR cannot read pictures or mathematics). Page furniture (header, footer, page number) is listed at the end. The same content is available as JSON with every box, label and text.

Language	Right script	Detected as
Assamese	10/10	Bengali 10
Bengali	10/10	Bengali 10
English	8/10	Bengali 1, Latin 8, Oriya 1
Gujarati	9/10	Gujarati 9, Latin 1
Hindi	9/10	Devanagari 9, Latin 1
Kannada	9/10	Kannada 9, Oriya 1
Malayalam	8/10	Devanagari 1, Latin 1, Malayalam 8
Marathi	6/10	Devanagari 6, Latin 2, Telugu 2
Odia	9/10	Gujarati 1, Oriya 9
Punjabi	10/10	Gurmukhi 10
Tamil	9/10	Kannada 1, Tamil 9
Telugu	9/10	Latin 1, Telugu 9

OCR takes about 8 seconds per page on this laptop. IndicDLP contains no transcriptions of the text, so character-level accuracy could not be measured; quality varies with print and scan quality and is usually good for clean printed pages. Two real examples of the output, unedited:

English test page `mn_en_000887_1.png`, detected script Latin, Tesseract languages `eng`:

- If the system came equipped with an internal embedded wireless device, do not operate the wireless device unless all covers and shields are in place and the system is fully assembled.
- Wireless devices are not user serviceable. Do not modify them in any way. Modification to a wireless device will void the authorization to use it. Please contact the manufacturer for service.
- Only use drivers approved for country in which the device will be used. See the manufacturer System Restoration Kit, or contact the manufacturer's technical support for additional information.

United States of America

USA and Canada Safety Requirements and Notices

Do not touch or move antenna while the unit is transmitting or receiving.

Do not hold any component containing the radio such that the antenna is very close to touching any exposed parts of the body, especially the face or eyes, while transmitting.

Do not operate the radio or attempt to transmit data unless the antenna is connected; if not, the radio may be damaged.

Use in specific environments:

The use of wireless devices in hazardous locations is limited by the constraints posed by the safety directors of such environments.

The use of wireless devices on airplanes is governed by the Federal Aviation Administration (FAA).

The use of wireless devices in hospitals is restricted to the limits set forth by each hospital.

- folio: Appendix
- page-number: 128

Hindi test page `sy_hi_000308_0.png`, detected script Devanagari, Tesseract languages `hin+mar+eng`:

आधारधूत संकल्पनाएँ : रूपविज्ञान का विषय क्षेत्र और प्रकृति; रूपिम, रूप, उपरूप, शून्य उपरूप, उपरूपों पर प्रतिबंध की संकल्पनाएँ; शब्दिम और शब्द; रूपिमों के प्रकार - स्वतन्त्र और बद्ध; धातु, अंग, आधार रूप, परप्रत्यय, मध्य प्रत्यय, पूर्व प्रत्यय, संवृक्त रूपिम, सर्वदेशी, आदिष्ट; प्रत्यय बनाम युक्त; व्याकरणिक संवर्ग- काल, पक्ष, वृत्ति, पुरुष, लिंग, वचन, विभक्ति चिहनक तथा विभक्ति संबंध; पूर्व और पश्च स्थान, रूपवैज्ञानिक विवरण के माडलः मद तथा विन्यास, मद या प्रक्रम, शब्द तथा रूपावली रूपवैज्ञानिक विश्लेषण: रूपिम की पहचान; रूपक्रियात्मक परिवर्तन; रूपस्वनिमिक प्रक्रियाएँ; अन्त: तथा बहि: संधि; विभक्ति तथा व्युत्पत्ति; धातु रूप तथा संज्ञा रूप।

अन्द-निमाणि प्रक्रियाएँ : व्युत्पत्ति (प्राथमिक बनाम द्वितीयक व्युत्पत्ति, नामिकीकरण, क्रियाकरण इत्यादि), समासीकरण (समासों के प्रकार: अन्तःकेंद्रिक, बहिःकेंद्रिक इत्यादि), पुनर्द्िगुणन, सादृश्यमूलक व्युत्पत्ति, परिवर्तन, छिन्नन, मिश्रण, परिवर्णी शब्द, लौकिक व्युत्पत्ति, सर्जनात्मकतातथा उत्पादकता , अवरोध, कोष्टन विरोधाभास, प्रत्यय क्रम-स्थापन में बाध्यताएँ।

रूप-वाक्य [FIT ८ नामिकीकरण तथा शब्दरचनागत प्राक्कल्पना; व्याकरणिक प्रकार्य परिवर्तनकारक नियम; प्रेरणार्थक, कर्मवाच्य।

इकाई-4 : वाक्य विज्ञान

पारंपरिक तथा संरचनात्मक वाक्याविज्ञान © शब्द T=: ATT वर्गीकरण (नाम, आख्यात, उपसर्ग, निपात); आधारिक वाक्यविन्यासी इकाइयाँ और उनके प्रकार: शब्द, पदबंध, उपवाक्य, वाक्य, कारक संबंध; व्याकरणिक संबंध तथा विभक्ति संबंध, रचना प्रकार (बहि: केंद्रिक, अन्तःकेंद्रिक इत्यादि), aa संघटक विश्लेषण

प्रजनक FFE; पैरामीटर तथा सर्वभौम व्याकरण, 'नल सब्जेक्ट पैरामीटर" , अन

8. Making it a working product

- **Web app** (Streamlit): upload an image or PDF (any page) and see it with coloured boxes. Move the confidence slider to show more or fewer parts instantly. Click *Read text* to get the text of every box (auto-detected or chosen language), and download it as Markdown or JSON. Other tabs show the dashboard, model comparison, evaluation charts and per-language results.
- **REST API** (FastAPI): other software sends a page to `POST /analyze` (optionally with `ocr=true`) and gets boxes, labels and text back as JSON.

Both run from **one Docker image** with all three models and the OCR engine inside. It is CPU-only, runs as a non-root user, has health checks and upload limits, and works with networking switched off. One command starts everything: `docker compose up --build`.

The project was audited end to end. 30 bugs were found and fixed, including the following:

- model loading that always crashed;
- YOLO training that had never worked;
- evaluation charts that were randomly generated;
- class names that did not match the dataset;
- a model that silently downloaded weights at run time;
- a sidebar that could not be reopened.

The test suite grew from 27 shape checks to 143 tests that check behaviour, and every fix was also verified on real data. The full list is in `docs/FINAL_AUDIT.md`.

9. Limitations and next steps

- **Our own models** were trained on 2293 pages for one or two hours on a laptop. Longer training on the full dataset with a cloud GPU is how they would approach the authors' model.
- **Rare labels** (`subsub-unordered-list`: 0 training boxes, `index`: 6 training boxes, `sub-unordered-list`: 13 training boxes, `table-of-contents`: 32 training boxes) have very few training examples, and all models handle them poorly.
- **OCR quality** could not be measured without transcriptions. Handwriting, very old prints and decorative fonts will read badly.
- **Out-of-domain pages:** IndicDLP covers 12 kinds of document; others (CVs, invoices, slides) may be segmented less well.
- **Reading order** is a simple top-to-bottom, left-to-right rule; complex newspaper layouts can come out jumbled.
- **Public deployment** needs a reverse proxy for HTTPS, login and rate limiting.

10. References

1. O. Nath, S. Kukkala, M. Khapra, R. K. Sarvadevabhatla. IndicDLP: A Foundational Dataset for Multi-lingual and Multi-domain Document Layout Parsing. ICDAR 2025, Springer LNCS, pp. 23-39. Models: Zenodo

record 15881917.
2. Z. Zhao, H. Kang, B. Wang, C. He. DocLayout-YOLO: Enhancing Document Layout Analysis through Diverse Synthetic Data and Global-to-Local Adaptive Perception. arXiv:2410.12628, 2024.
3. R. Smith. An Overview of the Tesseract OCR Engine. ICDAR 2007.
4. S. Ren, K. He, R. Girshick, J. Sun. Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. NeurIPS 2015.
5. G. Jocher, A. Chaurasia, J. Qiu. Ultralytics YOLOv8. 2023. github.com/ultralytics/ultralytics
6. T.-Y. Lin et al. Microsoft COCO: Common Objects in Context. ECCV 2014.
7. K. He, X. Zhang, S. Ren, J. Sun. Deep Residual Learning for Image Recognition. CVPR 2016.
8. T.-Y. Lin et al. Feature Pyramid Networks for Object Detection. CVPR 2017.